

SIMATS ENGINEERING
ASSESSMENT (COLLECTIONS)

COURSE CODE: CSA0904

NAME: Rose Mystica. J

COURSE NAME: Programming in Java

REG NO: 192421410

Q1 Library Book Management System

10 marks

Problem Statement:

You are developing a Library Management System where books are stored in a HashSet to ensure no duplicate books (based on ISBN). The system needs to:

1. Add books with attributes: ISBN, title, author, and publication year
2. Remove a book by its ISBN
3. Display all books sorted by publication year (newest first)
4. Find all books by a specific author

Requirements:

- Use HashSet<Book> for storing books
- Implement Iterator to traverse and remove books
- Create a custom Comparator for sorting
- Display books using iterator

Your Code

```
1 import java.util.*;
2 public class Book {
3     String title, author;
4     int year;
5     Book(String title, String author, int year) {
6         this.title = title;
7         this.author = author;
8         this.year = year;
9     }
10    public static void main(String[] args) {
11
12        HashSet<Book> set = new HashSet<>();
13
14        set.add(new Book("Java", "James", 2023));
15        set.add(new Book("Python", "Guido", 2022));
16        set.add(new Book("C++", "James", 2024));
17
18        ArrayList<Book> list = new ArrayList<>(set);
19
20        Collections.sort(list, new Comparator<Book>() {
21            public int compare(Book b1, Book b2) {
22                return b2.year - b1.year;
23            }
24        });
25        System.out.println("Books Sorted by Year:");
26        Iterator<Book> it = list.iterator();
27        while (it.hasNext()) {
```

```

24     },
25     System.out.println("Books Sorted by Year:");
26     Iterator<Book> it = list.iterator();
27     while (it.hasNext()) {
28         Book b = it.next();
29         System.out.println(b.title + " " + b.year);
30     }
31     System.out.println("\nBooks by James:");
32     for (Book b : list) {
33         if (b.author.equals("James")) {
34             System.out.println(b.title);
35         }
36     }
37 }
38 }
39

```

Input

```

Title: Java
Author: James
Year: 2023

```

Output

```

Books Sorted by Year:
C++ 2024
Java 2023
Python 2022
Books by James:

```

Output

```

Python 2022

Books by James:
C++
Java

```

Problem Statement:

A school maintains student records with their grades in different subjects. You need to analyze and process this data using ArrayList and HashMap.

1. Store students in an ArrayList<Student> where Student has: ID, name, and a Map<String, Integer> (subject → grade)
2. Calculate the average grade for each student
3. Find the top 3 students with highest average grades
4. Remove students who have failed in more than 2 subjects (grade < 40)

Requirements:

- Use ArrayList<Student> for student list
- Use HashMap<String, Integer> for grades within each student
- Use Iterator to traverse and remove students
- Implement sorting using Collections.sort() with custom comparator

Your Code

```
1 import java.util.*;
2 public class Student {
3     int id;
4     String name;
5     HashMap<String, Integer> marks = new HashMap<>();
6     Student(int id, String name) {
7         this.id = id;
8         this.name = name;
9     }
10    double avg() {
11        int sum = 0;
12        for (int m : marks.values())
13            sum += m;
14        return (double) sum / marks.size();
15    }
16    public static void main(String[] args) {
17
18        ArrayList<Student> list = new ArrayList<>();
19
20        Student s1 = new Student(1, "Rose");
21        s1.marks.put("Java", 90);
22        s1.marks.put("DBMS", 80);
23
24        Student s2 = new Student(2, "John");
25        s2.marks.put("Java", 70);
26        s2.marks.put("DBMS", 85);
27    }
```

```

27
28     Student s3 = new Student(3, "David");
29     s3.marks.put("Java", 95);
30     s3.marks.put("DBMS", 90);
31
32     list.add(s1);
33     list.add(s2);
34     list.add(s3);
35
36     Collections.sort(list, new Comparator<Student>() {
37         public int compare(Student a, Student b) {
38             return Double.compare(b.avg(), a.avg());
39         }
40     });
41     System.out.println("Student Averages:");
42     for (Student s : list) {
43         System.out.println(s.id + " " + s.name + " Average = " + s.avg());
44     }
45     System.out.println("\nTop 3 Students:");
46     for (int i = 0; i < Math.min(3, list.size()); i++) {
47         Student s = list.get(i);
48         System.out.println((i + 1) + ". " + s.name + " - " + s.avg());
49     }
50 }
51 }

```

Output

```

Student Averages:
3 David Average = 92.5
1 Rose Average = 85.0
2 John Average = 77.5

Top 3 Students:

```

```

Top 3 Students:
1. David - 92.5
2. Rose - 85.0
3. John - 77.5

```

Q3 Navigation History (Web Browser)

1 mark

Problem Statement:

You need to implement a web browser's navigation system where users can go back and forward through pages using a Stack.

1. Store visited URLs in a stack
2. Implement goBack() - moves one page back (but keeps forward history)
3. Implement goForward() - moves one page forward
4. Implement visitPage(String url) - adds a new page and clears forward history
5. Display current page and navigation history

Requirements:

- Use Stack<String> for back history
- Use Stack<String> for forward history
- Use Iterator to display all pages in both stacks without modifying them
- Handle edge cases (empty stack, end of navigation)

Your Code

```
1 import java.util.*;  
2 public class NavigationHistory {  
3     public static void main(String[] args) {  
4  
5         Stack<String> back = new Stack<>();  
6         Stack<String> forward = new Stack<>();  
7  
8         back.push("Google");  
9         back.push("YouTube");  
10        back.push("GitHub");  
11        System.out.println("Current Page: " + back.peek());  
12        if (back.size() > 1) {  
13            forward.push(back.pop());  
14        }  
15        System.out.println("After Go Back: " + back.peek());  
16        if (!forward.isEmpty()) {  
17            back.push(forward.pop());  
18        }  
19        System.out.println("After Go Forward: " + back.peek());  
20  
21        back.push("Wikipedia");  
22        forward.clear();  
23  
24        System.out.println("\nBack History:");  
25        Iterator<String> it = back.iterator();  
26        while (it.hasNext()) {  
27            System.out.println(it.next());  
28        }  
29    }  
30 }
```



```

20
21     back.push("Wikipedia");
22     forward.clear();
23
24     System.out.println("\nBack History:");
25     Iterator<String> it = back.iterator();
26     while (it.hasNext()) {
27         System.out.println(it.next());
28     }
29     System.out.println("\nForward History:");
30     Iterator<String> it2 = forward.iterator();
31     while (it2.hasNext()) {
32         System.out.println(it2.next());
33     }
34 }
35 }

```

Output

```

Current Page: GitHub
After Go Back: YouTube
After Go Forward: GitHub

Back History:
Google

```

```

Back History:
Google
YouTube
GitHub
Wikipedia

```

Problem Statement:

A company maintains employee records with their department and salary. You need to create a system that:

1. Store employees in a `HashMap<String, List<Employee>>` (Department → List of Employees)
2. Each Employee has: ID, name, department, salary
3. Calculate total salary expense per department
4. Find the highest-paid employee in each department
5. Remove employees with salary less than a threshold across all departments
6. Move an employee from one department to another

Requirements:

- Use `HashMap<String, List<Employee>>` for departmental storage
- Use `Iterator` to traverse both the map entries and the lists
- Use `Iterator` for safe removal while iterating
- Create methods to display department-wise statistics

Your Code

```
1 import java.util.*;
2 public class Employee {
3     int id;
4     String name;
5     double salary;
6     Employee(int id, String name, double salary) {
7         this.id = id;
8         this.name = name;
9         this.salary = salary;
10    }
11    public static void main(String[] args) {
12
13        HashMap<String, List<Employee>> map = new HashMap<>();
14
15        List<Employee> it = new ArrayList<>();
16        it.add(new Employee(1, "Rose", 50000));
17        it.add(new Employee(2, "John", 30000));
18
19        List<Employee> cse = new ArrayList<>();
20        cse.add(new Employee(3, "David", 45000));
21
22        map.put("IT", it);
23        map.put("CSE", cse);
24
25        for (String dept : map.keySet()) {
26            double total = 0;
27            Employee high = map.get(dept).get(0);
```

```

24
25     for (String dept : map.keySet()) {
26         double total = 0;
27         Employee high = map.get(dept).get(0);
28
29         Iterator<Employee> itr = map.get(dept).iterator();
30         while (itr.hasNext()) {
31             Employee e = itr.next();
32             total += e.salary;
33             if (e.salary > high.salary)
34                 high = e;
35         }
36         System.out.println("Department : " + dept);
37         System.out.println("Total Salary : " + total);
38         System.out.println("Highest Paid : " + high.name);
39         System.out.println();
40     }
41 }
42 }

```

Output

```

Department : CSE
Total Salary : 45000.0
Highest Paid : David

```

```

Department : IT
Total Salary : 80000.0

```

```

Department : IT
Total Salary : 80000.0
Highest Paid : Rose

```